Discriminative Learning: Linear Classifiers I

Least squares, perceptron, and logistic regression

Kangning Cui
City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning objectives

Three goals for this lecture:

- Build and interpret a linear classification boundary.
- Learn least-squares, perceptron, and logistic models.
- Calculate updates and choose losses for real tasks.

A linear decision rule can be trained using different objectives.

Linear decision functions

CS5487 ▪ Semester A, 2026/27

From density to boundary

Previous approach: model the class distributions $p(\boldsymbol{x} \mid y)$. **This week:** learn a rule that separates labeled examples directly.

$$\boxed{\boldsymbol{x} \longrightarrow \text{linear score} \longrightarrow \text{predicted class}}$$

- Example: classify an email from counts of suspicious phrases.
- We may not need to model how every possible email is generated.

Linear score

For an input $\boldsymbol{x} \in \mathbb{R}^d$, define:

$$\boxed{f(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b}$$

- d: number of features; $\boldsymbol{w} \in \mathbb{R}^d$: feature weights.
- $b \in \mathbb{R}$: intercept; $f(\boldsymbol{x})$: real-valued score.
- For $y \in \{-1, +1\}$, predict by the sign of f.

The decision boundary is $f(\boldsymbol{x}) = 0$. We will treat a zero score as a boundary tie unless a rule is specified.

Two-feature prediction

Consider an illustrative spam filter:

- x_1: suspicious phrase count; x_2: trusted phrase count.
- Fitted score: $f(\boldsymbol{x}) = x_1 - 2x_2 + 1$.

Email	(x_1, x_2)	Score	Prediction
A	$(3, 1)$	$3 - 2 + 1 = 2$	Spam $(+1)$
B	$(1, 2)$	$1 - 4 + 1 = -2$	Not spam (-1)

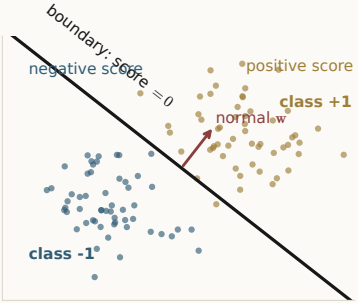
The sign sets the decision; weights control each feature's contribution. *Illustrative parameters, not measured email statistics.*

Boundary geometry

The boundary satisfies:

$$\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b = 0.$$

- $\boldsymbol{w}$: normal to the boundary.
- b: moves the boundary.
- In 2D it is a line; in d dimensions, a hyperplane.



Scaling the score

Multiply the weight and intercept by the same $c > 0$:

$$f'(\boldsymbol{x}) = c\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + cb = cf(\boldsymbol{x}).$$

Parameter scaling	Score	Decision
Original	2	$+1$
$c = 3$	6	$+1$

- The separator and predicted labels do not change.
- Raw score magnitude alone is not a probability or confidence measure.

Signed distance

The *score* and the *perpendicular distance* are not the same. For $\boldsymbol{w} \neq \mathbf{0}$:

$$\boxed{d_{\text{signed}}(\boldsymbol{x}) = \frac{f(\boldsymbol{x})}{\|\boldsymbol{w}\|_2}}$$

For $\boldsymbol{w} = (3, 4)^{\mathsf{T}}$, $b = -5$, and $\boldsymbol{x} = (3, 1)^{\mathsf{T}}$:

$$f(\boldsymbol{x}) = 8, \qquad \|\boldsymbol{w}\|_2 = 5, \qquad d_{\text{signed}} = \boxed{1.6}.$$

Multiplying $\boldsymbol{w}, b$ by 3 changes the score to 24, but not the distance: $24/15 = 1.6$.

Intercept

Add a constant 1 to each input to include the intercept in the weights:

$$\tilde{\boldsymbol{x}} = \begin{bmatrix} \boldsymbol{x} \\ 1 \end{bmatrix}, \qquad \tilde{\boldsymbol{w}} = \begin{bmatrix} \boldsymbol{w} \\ b \end{bmatrix}.$$

Then $f(\boldsymbol{x}) = \tilde{\boldsymbol{w}}^{\mathsf{T}} \tilde{\boldsymbol{x}}$.

- This gives a shorter notation for later derivations.
- The intercept lets the boundary move away from the origin.
- It is usually excluded from weight regularization.

Signed score

For a true label $y \in \{-1, +1\}$, define the **signed score**:

$$\boxed{m = y f(\boldsymbol{x})}.$$

True label	Score	m	Interpretation
$+1$	2	2	Correct
-1	2	-2	Incorrect
-1	-3	3	Correct

A positive m means the observation lies on the correct side. Many classification losses use this single quantity.

Learning the parameters

A linear score defines a possible boundary, but its weights and intercept must still be learned from labeled data.
Consider two examples: $(-2, -1)$ and $(2, +1)$, written as (x, y).

Candidate score	Prediction at $x = -2$	Prediction at $x = 2$
$f_A(x) = x$	-1 (correct)	$+1$ (correct)
$f_B(x) = -x$	$+1$ (wrong)	-1 (wrong)

What is missing: a training objective that favors parameters matching the observed labels. Least squares provides a first approach.

Least squares

CS5487 ▪ Semester A, 2026/27

Least-squares classifier

Motivation: can we use linear regression to predict class labels? Use $y_i \in \{-1, +1\}$ and fit the score by squared error:

$$J(\boldsymbol{w}, b) = \sum_{i=1}^{n} (y_i - \boldsymbol{w}^\mathsf{T} \boldsymbol{x}_i - b)^2$$

- $(\boldsymbol{x}_i, y_i)$: labeled training observation.
- n: number of training observations.
- Classify using the sign of the fitted score.

Limitation: squared error measures numeric disagreement, not just whether the predicted class is correct.

Closed-form solution

Put the intercept in the design matrix $\boldsymbol{X}_a \in \mathbb{R}^{n \times (d+1)}$:

$$\boldsymbol{X}_a = \begin{bmatrix} \boldsymbol{x}_1^\mathsf{T} & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_n^\mathsf{T} & 1 \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \boldsymbol{w} \\ b \end{bmatrix}.$$

With labels $\boldsymbol{y} = (y_1, \ldots, y_n)^\mathsf{T}$, minimize $J(\boldsymbol{\beta}) = \|\boldsymbol{y} - \boldsymbol{X}_a\boldsymbol{\beta}\|_2^2$.

$$\nabla_\beta J = -2\boldsymbol{X}_a^\mathsf{T}(\boldsymbol{y} - \boldsymbol{X}_a\boldsymbol{\beta}) = 0$$

$$\boxed{\boldsymbol{X}_a^\mathsf{T}\boldsymbol{X}_a\,\hat{\boldsymbol{\beta}} = \boldsymbol{X}_a^\mathsf{T}\boldsymbol{y}}$$

These **normal equations** can be solved as a linear system.

Least-squares data

Fit a one-feature score $f(x) = wx + b$ to four labeled examples:

x	-1	0	1	2
y	-1	-1	$+1$	$+1$

For the normal equations, compute four sums:

$$\sum x_i = 2, \qquad \sum x_i^2 = 6,$$
$$\sum y_i = 0, \qquad \sum x_i y_i = 4.$$

These numbers determine the 2-by-2 system for w and b.

Least-squares fit

The normal equations for the four observations are:

$$\begin{bmatrix} 6 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} w \\ b \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix}.$$

Equivalently:

$$6w + 2b = 4, \qquad 2w + 4b = 0.$$

From the second equation, $b = -w/2$. Substitution gives:

$$5w = 4 \quad \Longrightarrow \quad \boxed{w = 0.8, \qquad b = -0.4}.$$

The learned rule is $f(x) = 0.8x - 0.4$.

Least-squares predictions

Use $f(x) = 0.8x - 0.4$ and predict by its sign:

x	True y	Score	Predicted y
-1	-1	-1.2	-1
0	-1	-0.4	-1
1	$+1$	0.4	$+1$
2	$+1$	1.2	$+1$

All four predictions are correct. Yet the squared loss is:

$$J = 0.04 + 0.36 + 0.36 + 0.04 = \boxed{0.8}.$$

The boundary is at $x = 0.5$, where $f(x) = 0$.

Loss mismatch

Least squares tries to match the numerical target $y = +1$:

Predicted score	Correct class	Squared loss $(1 - f)^2$
$f = 1$	Yes	0
$f = 5$	Yes	16
$f = -1$	No	4

- A very confident correct classification can receive a large penalty.
- We therefore want a loss that reacts to an incorrect class, not merely to distance from the numeric target $+1$ or -1.

Perceptron

CS5487 ▪ Semester A, 2026/27

Perceptron idea

Least squares fits numeric labels, even when the class decision is correct. The perceptron instead learns from wrong-side predictions. For labels $y_i \in \{-1, +1\}$, compute:

$$m_i = y_i(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b).$$

- $m_i > 0$: correct side of the boundary; do nothing.
- $m_i \leq 0$: mistake or tie; update the parameters.

The perceptron is an **online** method: it can process labeled examples one at a time.

Perceptron loss

Use the signed score $m_i = y_i f(\boldsymbol{x}_i)$:

$$\boxed{\ell_i = \max(0, -m_i)}.$$

m_i	Loss	Meaning
2	0	Correct side
-1	1	Wrong side
-3	3	Larger mistake

At $m_i = 0$ the loss is nondifferentiable. We choose to update at ties as well as mistakes.

Update derivation

For a mistaken example, the loss is:

$$\ell_i = -y_i(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b).$$

Its gradients are:

$$\nabla_{\boldsymbol{w}}\ell_i = -y_i\boldsymbol{x}_i, \qquad \frac{\partial \ell_i}{\partial b} = -y_i.$$

A step of gradient descent with learning rate $\eta > 0$ gives:

$$\boxed{\begin{aligned} \boldsymbol{w} &\leftarrow \boldsymbol{w} + \eta y_i \boldsymbol{x}_i, \\ b &\leftarrow b + \eta y_i. \end{aligned}}$$

Apply this update only when $m_i \leq 0$.

One update

Start from $\boldsymbol{w} = (0,0)^\mathsf{T}$, $b = 0$, and $\eta = 1$. The first example has $\boldsymbol{x}_1 = (1,2)^\mathsf{T}$ and $y_1 = +1$.

$$f(\boldsymbol{x}_1) = 0 \qquad \text{(boundary tie: update)}.$$

$$\boldsymbol{w} \leftarrow (0,0)^\mathsf{T} + (1,2)^\mathsf{T} = \boxed{(1,2)^\mathsf{T}},$$
$$b \leftarrow 0 + 1 = \boxed{1}.$$

The new score is $1 + 4 + 1 = \boxed{6 > 0}$. This positive example is now classified correctly.

After the update

Current parameters: $\boldsymbol{w} = (1, 2)^\mathsf{T}$, $b = 1$. The next example is $\boldsymbol{x}_2 = (2, 0)^\mathsf{T}$, $y_2 = -1$.

$$f(\boldsymbol{x}_2) = 2 + 1 = 3, \qquad m_2 = (-1)(3) = -3 < 0.$$

The prediction is wrong. With $\eta = 1$:

$$\boldsymbol{w} \leftarrow (1, 2)^\mathsf{T} - (2, 0)^\mathsf{T} = \boxed{(-1, 2)^\mathsf{T}},$$
$$b \leftarrow 1 - 1 = \boxed{0}.$$

The new score for the negative example is $-2 < 0$.

Two-point pass

After the two updates: $\boldsymbol{w} = (-1, 2)^{\mathsf{T}}$, $b = 0$.

Training point	True y	New score	yf
$\boldsymbol{x}_1 = (1, 2)$	$+1$	$-1 + 4 = 3$	$3 > 0$
$\boldsymbol{x}_2 = (2, 0)$	-1	-2	$2 > 0$

Both observations are now on the correct side. A new pass over *these two* examples produces no update. **Use:** simple sequential learning without fitting a probability distribution.

Convergence

The perceptron has a useful guarantee when the training data are **linearly separable** with a positive margin and bounded features:

- It makes only finitely many mistakes.
- The final separator need not be unique.
- Sample order can change the boundary obtained.

When labels overlap, updates may continue indefinitely. **Practice:** cap the number of passes and monitor validation accuracy. Logistic regression instead fits a class *probability*.

Probability estimates for decisions

The perceptron learns a separator: a positive score predicts class $+1$. Its score is *not* an estimated probability.

- Scores 2 and 20 both predict $+1$; neither means "20%" or "200% probability."
- In fraud screening, two positive cases may have different priorities.
- Unequal error costs may require a threshold on estimated risk.

What is missing: a model that maps a linear score to a class probability and learns that probability from labeled outcomes. This motivates logistic regression.

Logistic regression

CS5487 ▪ Semester A, 2026/27

Sigmoid

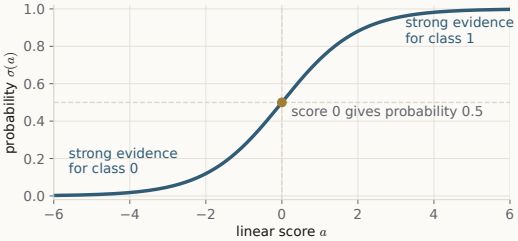
A linear separator supplies a score, but risk-based decisions may need estimated probabilities. The sigmoid maps any real score to $(0, 1)$.

$$\boxed{p(y = 1 \mid \boldsymbol{x}) = \sigma(a)}$$

where

$$a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b, \qquad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Here labels are $y \in \{0, 1\}$.



Score and probability

The sigmoid transforms a linear score a into a positive-class probability.

Score a	-2	-1	0	1
Probability $\sigma(a)$	0.119	0.269	0.500	0.731

$$a > 0 \iff p(y = 1 \mid \boldsymbol{x}) > 0.5$$

- $a = 0$ gives the default 50% classification boundary.
- Rescaling the score changes its predicted probability, even when the class is unchanged.

Log-odds

Let $p = p(y = 1 \mid \boldsymbol{x})$. The **odds** compare the probabilities of the two classes:

$$\text{odds} = \frac{p}{1-p}.$$

For $p = \sigma(a)$, we have $p/(1-p) = e^a$, so:

$$\boxed{\log \frac{p}{1-p} = a = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b}.$$

Logistic regression assumes that **log-odds are linear** in the features. Probability itself is not a linear function of the features.

Risk prediction example

Consider an *illustrative* risk score:

$$a = 0.8x_1 - 0.4x_2 - 1,$$

where x_1 counts warning signals and x_2 protective signals.

Case	(x_1, x_2)	Score a	$\sigma(a)$
A	$(2, 1)$	0.2	0.550
B	$(0, 1)$	-1.4	0.198

The model assigns higher estimated risk to A. These example probabilities are not clinical estimates.

Fitting probabilities to labels

The sigmoid defines $p_i = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b)$, but it does not tell us *which* $\boldsymbol{w}$ and b to use.

Observed label	Positive-class probability p_i	Probability of observed label
$y_i = 1$	0.9	0.9
$y_i = 1$	0.1	0.1
$y_i = 0$	0.9	0.1

Training goal: choose parameters that assign large probabilities to the class labels actually observed. **Likelihood** measures this fit.

Bernoulli likelihood

Use binary labels $y_i \in \{0, 1\}$ and define:

$$a_i = \boldsymbol{w}^\mathsf{T} \boldsymbol{x}_i + b, \qquad p_i = \sigma(a_i).$$

The likelihood of the observed class label is:

$$\boxed{p(y_i \mid \boldsymbol{x}_i) = p_i^{y_i} (1 - p_i)^{1 - y_i}}.$$

- If $y_i = 1$, the contribution is p_i.
- If $y_i = 0$, it is $1 - p_i$.

This is the **conditional likelihood of the labels**, not the feature likelihood $p(\boldsymbol{x} \mid y)$ used by generative classifiers.

Goal: make the observed training labels probable.

Dataset likelihood

Assume the training labels are conditionally independent given their features. The dataset likelihood is:

$$L(\boldsymbol{w}, b) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Taking logarithms converts the product to a sum:

$$\boxed{\log L = \sum_{i=1}^{n} \big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \big]}.$$

The labels are observed; changing $\boldsymbol{w}, b$ changes their probabilities. We choose parameters that maximize this log-likelihood.

Likelihood compares models

Suppose the observed labels are $(y_1, y_2) = (1, 0)$. Two candidate models predict positive-class probabilities:

Model	(p_1, p_2)	Accuracy at 0.5	Likelihood
A	$(0.9, 0.1)$	$2/2$	$0.9(1 - 0.1) = 0.81$
B	$(0.6, 0.4)$	$2/2$	$0.6(1 - 0.4) = 0.36$

Both models achieve 100% accuracy on these examples, but A assigns higher probability to the observed outcomes. **Significance:** likelihood uses information that binary accuracy loses. Its logarithm also turns products over many examples into sums.

Cross-entropy

Likelihood says how well the model explains the observed labels. To train by minimization, take the *negative* log-likelihood. For one example, the **binary cross-entropy** (BCE) is:

$$\boxed{\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)}.$$

For the whole dataset:

$$J = \sum_{i=1}^{n} \ell_i, \qquad \bar{J} = \frac{1}{n} \sum_{i=1}^{n} \ell_i.$$

We use J for the summed loss and $\bar{J}$ for the average. The formula directly penalizes low probability assigned to the true class.

Dataset cross-entropy

Suppose $\boldsymbol{y} = (1, 0, 1)^\mathsf{T}$ and model probabilities $\boldsymbol{p} = (0.8, 0.25, 0.6)^\mathsf{T}$.

y_i	Probability of the true class	BCE ℓ_i
1	0.8	$-\log 0.8 = 0.223$
0	$1 - 0.25 = 0.75$	$-\log 0.75 = 0.288$
1	0.6	$-\log 0.6 = 0.511$

The average loss is:

$$\boxed{\bar{J} = (0.223 + 0.288 + 0.511)/3 \approx 0.341}.$$

This metric assesses probability quality, not just prediction accuracy.

From loss to parameter updates

We have a training objective, $J = \sum_i \ell_i$, that scores the fitted probabilities. We still need to *minimize it* over $\boldsymbol{w}$ and b.

Label	Predicted p_i	BCE	Helpful score change
$y_i = 1$	0.2	$-\log(0.2) = 1.609$	Increase a_i
$y_i = 0$	0.8	$-\log(0.2) = 1.609$	Decrease a_i

Here $a_i = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b$. There is no general least-squares-style linear system for logistic fitting.
Next: derivatives tell us how to update the score and parameters.

Sigmoid derivative

To update $\boldsymbol{w}$ and b, first measure how the probability $p = \sigma(a)$ changes when its score a changes. Starting from $\sigma(a) = 1/(1 + e^{-a})$:

$$\frac{dp}{da} = \frac{e^{-a}}{(1 + e^{-a})^2} = \boxed{p(1 - p)}.$$

At $p = 0.5$, the slope is 0.25. This identity is the first step in differentiating cross-entropy.

Score gradient

For one example, let $a_i = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b$ and $p_i = \sigma(a_i)$. Differentiate cross-entropy with respect to p_i:

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}.$$

Using $dp_i/da_i = p_i(1 - p_i)$ and the chain rule:

$$\boxed{\frac{\partial \ell_i}{\partial a_i} = p_i - y_i}.$$

The complicated expression simplifies to *prediction minus label*.

Parameter gradient

Because $a_i = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_i + b$:

$$\frac{\partial a_i}{\partial \boldsymbol{w}} = \boldsymbol{x}_i, \qquad \frac{\partial a_i}{\partial b} = 1.$$

Apply the chain rule to the result $\partial \ell_i / \partial a_i = p_i - y_i$:

$$\boxed{\begin{aligned} \nabla_{\boldsymbol{w}} \ell_i &= (p_i - y_i)\boldsymbol{x}_i, \\ \frac{\partial \ell_i}{\partial b} &= p_i - y_i. \end{aligned}}$$

The error term controls the direction; the input features scale the weight updates.

Gradient direction

Gradient descent subtracts the derivative of the loss.

True label	Predicted p_i	$p_i - y_i$
Positive $y_i = 1$	0.2	-0.8
Negative $y_i = 0$	0.8	$+0.8$

- Negative gradient for the positive case: *increase* its score.
- Positive gradient for the negative case: *decrease* its score.

Both updates favor the observed label.

Batch gradient

Let $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ store feature rows and $\boldsymbol{p}, \boldsymbol{y} \in \mathbb{R}^n$ store model probabilities and labels. For **average BCE**, $\bar{J} = \frac{1}{n} \sum_i \ell_i$:

$$\boxed{\begin{aligned} \nabla_{\boldsymbol{w}} \bar{J} &= \frac{1}{n} \boldsymbol{X}^\mathsf{T} (\boldsymbol{p} - \boldsymbol{y}), \\ \frac{\partial \bar{J}}{\partial b} &= \frac{1}{n} \sum_i (p_i - y_i). \end{aligned}}$$

Each training observation contributes an error $p_i - y_i$. Average gradients keep the update scale comparable across dataset sizes.

Mini-batch setup

To train on a subset of observations, average only their gradients. At $\boldsymbol{w} = \mathbf{0}$ and $b = 0$, every example has $p_i = 0.5$.

Example	$\boldsymbol{x}_i$	y_i	$p_i - y_i$
1	$(2, 0)^\mathsf{T}$	1	-0.5
2	$(0, 1)^\mathsf{T}$	0	$+0.5$

This mini-batch has $m = 2$ observations. We use $1/m$ (not $1/n$) for its mean gradient.

Mini-batch gradient

Using the two examples from the previous slide:

$$\nabla_{\boldsymbol{w}} \bar{J}_{\text{batch}} = \frac{1}{2}\left[(-0.5)\begin{bmatrix}2\\0\end{bmatrix} + (0.5)\begin{bmatrix}0\\1\end{bmatrix}\right]$$
$$= \boxed{\begin{bmatrix}-0.5\\0.25\end{bmatrix}}.$$

The intercept gradient is $(-0.5 + 0.5)/2 = 0$. With learning rate $\eta = 0.2$:

$$\boxed{\boldsymbol{w}_{\text{new}} = (0.1, -0.05)^{\mathsf{T}}, \qquad b_{\text{new}} = 0}.$$

A mini-batch approximates the full-dataset gradient at lower update cost.

One gradient step

Take one positive example:

$$\boldsymbol{x} = (2, -1)^{\mathsf{T}}, \quad y = 1, \qquad \boldsymbol{w} = (0, 0)^{\mathsf{T}},\ b = 0.$$

Initially $a = 0$ and $p = \sigma(0) = 0.5$. The BCE error is $p - y = -0.5$.

$$\nabla_{\boldsymbol{w}} \ell = (-0.5) \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \boxed{\begin{bmatrix} -1 \\ 0.5 \end{bmatrix}},$$

$$\frac{\partial \ell}{\partial b} = \boxed{-0.5}.$$

Next, apply one gradient-descent step with $\eta = 0.2$.

Updated parameters

Using $\eta = 0.2$, subtract the previous gradients:

$$\boxed{\boldsymbol{w}_{\text{new}} = (0.2, -0.1)^{\mathsf{T}}, \qquad b_{\text{new}} = 0.1}.$$

For the same positive example:

$$a_{\text{new}} = 0.2(2) + (-0.1)(-1) + 0.1 = 0.6,$$
$$p_{\text{new}} = \sigma(0.6) \approx \boxed{0.646}.$$

Its BCE falls from $-\log 0.5 = 0.693$ to $-\log 0.646 \approx \boxed{0.437}$. **Result:** the update raises the probability of the true class.

Convexity

For one example, the logistic loss is convex in the linear score a:

$$\boxed{\frac{d^2\ell}{da^2} = p(1-p) \geq 0, \qquad p = \sigma(a)}.$$

Since $a = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + b$ is linear in the parameters, the sum of logistic losses is also **convex** in $\boldsymbol{w}, b$.

- No spurious local minima in this objective.
- A minimizer need not be unique.
- Convexity does not guarantee good test accuracy.

This helps make logistic regression easier to optimize than many nonlinear models.

Perfect separation

Consider two observations: $(x, y) = (-1, 0)$ and $(1, 1)$. Use score $a = wx$, with $b = 0$ and any $w > 0$. Both labels are correctly classified. The **summed BCE** is:

$$\boxed{J(w) = 2\log(1 + e^{-w})}.$$

As $w \to \infty$, this loss approaches 0; no finite w reaches the limiting value. **Practical consequence:** unregularized logistic regression can keep increasing coefficient size on separable data.

Regularization

To control coefficient size, add an ℓ_2 penalty to average BCE:

$$\boxed{\bar{J}_\lambda(\boldsymbol{w}, b) = \bar{J}(\boldsymbol{w}, b) + \frac{\lambda}{2}\|\boldsymbol{w}\|_2^2}.$$

Here $\lambda \geq 0$ is the regularization strength. The weight gradient becomes:

$$\boxed{\nabla_{\boldsymbol{w}} \bar{J}_\lambda = \frac{1}{n}\boldsymbol{X}^\mathsf{T}(\boldsymbol{p} - \boldsymbol{y}) + \lambda \boldsymbol{w}}.$$

- Larger λ discourages large weights.
- The intercept b is not penalized here.

Choose λ using validation data.

Regularization in numbers

Return to the separable two-point example with $b = 0$. Its *average* BCE is $\bar{J}(w) = \log(1 + e^{-w})$. For $\lambda = 0.1$, add the penalty $0.05w^2$:

Weight w	Average BCE	Penalty	Total $\bar{J}_\lambda$
1	0.3133	0.0500	0.3633
2	0.1269	0.2000	0.3269
4	0.0181	0.8000	0.8181

All three weights classify both training examples correctly. **Meaning:** lower BCE does not always justify much larger weights. The regularized objective balances fit and coefficient size.

Beyond binary classification

Binary logistic regression models two mutually exclusive outcomes using p and $1 - p$.

- A news classifier may need one label among *sports*, *science*, *and arts*.
- Three separate sigmoid outputs such as $(0.8, 0.7, 0.6)$ sum to 2.1: they are not a probability distribution over exactly one class.
- We need class scores that *compete* and are normalized to sum to 1.

Softmax provides this multiclass probability model. Independent sigmoids remain useful for *multi-label* problems where several labels can be true together.

Multiclass scores

For $K \geq 2$ classes, use one linear score per class:

$$a_k = \boldsymbol{w}_k^{\mathsf{T}} \boldsymbol{x} + b_k, \qquad k = 1, \ldots, K.$$

The **softmax** converts the scores to probabilities:

$$\boxed{p(y = k \mid \boldsymbol{x}) = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}}}.$$

The K probabilities sum to 1. **Application:** a softmax head classifies image or text embeddings from a deep neural network.

Softmax example

Three class scores are $(a_1, a_2, a_3) = (2, 1, 0)$. Subtract the largest score for numerical stability:

$$(2, 1, 0) \;\longrightarrow\; (0, -1, -2).$$

Softmax gives:

$$\boldsymbol{p} = \frac{(1, e^{-1}, e^{-2})}{1 + e^{-1} + e^{-2}} \approx \boxed{(0.665, 0.245, 0.090)}.$$

The prediction is $\boxed{\text{class } 1}$. Subtracting the same constant from every score *does not change* the softmax probabilities.

Training objectives

CS5487 ▪ Semester A, 2026/27

Empirical risk

Least squares, the perceptron, and logistic regression use a similar linear score but judge their predictions with different losses. For parameters θ, model f_θ, and example loss ℓ:

$$\hat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(y_i, f_\theta(\boldsymbol{x}_i)\big).$$

- θ: all trainable parameters (such as $\boldsymbol{w}, b$).
- ℓ: loss for one labeled prediction.
- $\hat{R}_n$: mean training loss on n observations.

Regularization adds a penalty such as $\lambda \|\boldsymbol{w}\|_2^2/2$ to control complexity.

Classification losses

Use labels $y \in \{-1, +1\}$ and the signed score $m = yf(\boldsymbol{x})$.

Loss	Formula	Training preference
Zero–one	$\mathbb{I}[m \leq 0]$	Count wrong decisions
Perceptron	$\max(0, -m)$	Correct the wrong side
Logistic	$\log(1 + e^{-m})$	Penalize low true-class probability
Hinge	$\max(0, 1 - m)$	Encourage a margin

For logistic regression with $0/1$ labels, use $2y - 1$ to define m.

Loss values

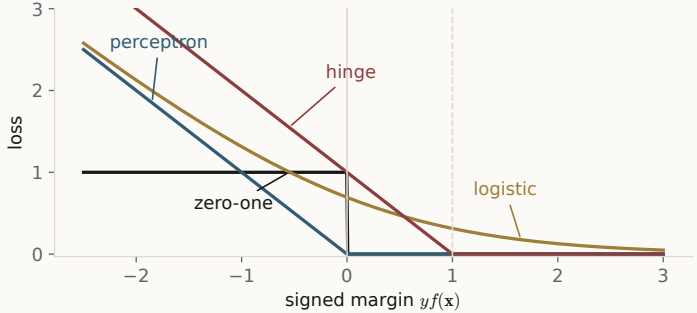
Compare losses at the same signed score $m = yf(\boldsymbol{x})$:

m	Zero–one	Perceptron	Logistic	Hinge
-1	1	1	1.313	2
0	1	0	0.693	1
1	0	0	0.313	0
2	0	0	0.127	0

- Perceptron loss stops penalizing as soon as $m \geq 0$.
- Logistic loss keeps decreasing as the correct-class score grows.

At $m = 0$, a perceptron update is a convention despite zero loss.

Loss curves



- Perceptron loss is zero on the correct side.
- Hinge loss becomes zero at $m \geq 1$.
- Logistic loss decreases smoothly with m.

Choosing a training loss

The *same linear score* can be trained for different goals.

Practical need	Reasonable starting point
Quick regression-style baseline	Least squares
Update from arriving labeled examples	Perceptron
Rank cases and estimate class probabilities	Logistic regression
Seek a separator with an explicit margin	Hinge loss / SVM (next week)

Practice: choose the objective for the task; then evaluate its predictions on held-out data.

Gradient strategies

An **epoch** processes the training examples once. Suppose the dataset has $2{,}000$ observations.

Method	Examples per update	Updates per epoch
Full batch	$2{,}000$	1
Stochastic	1	$2{,}000$
Mini-batch	50	40

Small batches are cheaper per update but noisier. Mini-batches often give a useful compromise.

Learning rate

Gradient descent uses $\theta \leftarrow \theta - \eta \nabla J(\theta)$, where $\eta > 0$ is the **learning rate**. Consider $J(\theta) = \theta^2/2$ and start at $\theta_0 = 2$.

η	New $\theta_1 = 2 - 2\eta$	New loss
0.5	1	0.5
3	-4	8

The larger step *increases* the loss. A correct gradient direction does not guarantee a good step size.

Feature scaling

For logistic regression, one weight gradient is:

$$\nabla_{\boldsymbol{w}} \ell_i = (p_i - y_i)\boldsymbol{x}_i.$$

If one feature is around 0.01 and another around $10{,}000$, the gradient contributions can have very different scales.

- Standardization often improves optimization.
- Fit the scaler on *training data only*.
- Interpret coefficients in the standardized units.

Training vs. validation

Illustrative training and validation losses:

Epoch	Training loss	Validation loss
10	0.45	0.50
20	0.25	0.30
30	0.12	0.38

Training loss keeps falling, but validation loss is lowest at epoch 20. **Practice:** use validation data for early stopping or hyperparameter selection. Keep test data for final evaluation.

Logistic implementation

```
import numpy as np
from scipy.special import expit # stable sigmoid

w = np.zeros(X_train.shape[1])
b = 0.0
lr, lam, steps = 0.1, 0.01, 500
for _ in range(steps):
    p = expit(X_train @ w + b)
    grad_w = X_train.T @ (p-y_train) / len(y_train)
    grad_w += lam * w
    grad_b = np.mean(p-y_train)
    w -= lr * grad_w
    b -= lr * grad_b
```

X_{train} stores feature rows; $y_{\text{train}} \in \{0,1\}^n$. The code minimizes average BCE plus an ℓ_2 weight penalty.

From training to decisions

Optimization gives fitted parameters and predicted class probabilities. A useful classification system still needs decisions about deployment.

- **Interpretation:** what does changing one feature do to the odds?
- **Actions:** should every case with $p > 0.5$ be treated as positive?
- **Validation:** do probabilities and error rates hold on new cases?

For example, $p(y = 1 \mid \boldsymbol{x}) = 0.35$ does not by itself specify whether a case needs a follow-up review; the error costs matter. **Next:** interpret coefficients, choose a threshold, and examine limits.

Interpretation

CS5487 ▪ Semester A, 2026/27

Coefficient meaning

A fitted model should be interpretable as well as predictive. Logistic regression models:

$$\log \frac{p}{1-p} = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b, \qquad p = p(y = 1 \mid \boldsymbol{x}).$$

Holding the other features fixed, increasing x_j by one unit:

$$\boxed{\text{log-odds change} = w_j}, \qquad \boxed{\text{odds multiplier} = e^{w_j}}.$$

A positive w_j raises the fitted odds of class 1; a negative coefficient lowers them. **Caution:** association is not proof of causation.

Odds example

Suppose $w_j = 0.7$ and the other features do not change. Increasing x_j by one unit multiplies the odds by:

$$\boxed{e^{0.7} \approx 2.014}.$$

If the original probability is $p = 0.5$, the odds are 1. New odds are 2.014, giving:

$$p_{\text{new}} = \frac{2.014}{1 + 2.014} \approx \boxed{0.668}.$$

The *odds* roughly double, but the *probability* does not.

Decision threshold

A fitted logistic model produces $p = p(y = 1 \mid \boldsymbol{x})$. The **decision threshold** τ is a separate choice:

$$\hat{y} = \begin{cases} 1, & p > \tau, \\ 0, & p \leq \tau. \end{cases}$$

- Lower τ: more positive predictions, usually more false alarms.
- Higher τ: fewer false alarms, usually more missed positives.

Choose τ using validation data and application-specific costs.

Threshold and errors

Suppose three cases receive probabilities $(0.15, 0.35, 0.70)$.

Threshold	Flagged probabilities	Cases flagged
0.5	0.70	1
0.2	$0.35,\ 0.70$	2

If a false negative costs 4 and a false positive costs 1, under the usual zero-cost-correct-decisions model:

$$\boxed{\tau^* = \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}} = \frac{1}{1+4} = 0.2}.$$

This rule assumes calibrated posterior probabilities and the stated costs.

Interpretation limits

A logistic coefficient measures an **association** in the fitted model. It is not automatically a causal effect.

- **Confounding:** an omitted variable affects both features and outcome.
- **Correlated inputs:** separate coefficients can be unstable.
- **Dataset shift:** deployment cases differ from training cases.

Example: a hospital test associated with disease does not mean that taking the test causes the disease.

Model comparison

All three methods fit a linear score but use different training signals.

Method	What is optimized	Output
Least squares	Squared error to numeric labels	Score
Perceptron	Wrong-side classification scores	Score
Logistic regression	Label likelihood / BCE	Probability

The boundary may be linear in all three cases. **The loss changes what the learning algorithm prefers.**

Linear probes for embeddings

A pretrained Transformer can represent a sentence with an embedding $\boldsymbol{h} \in \mathbb{R}^{768}$. Freeze the Transformer and fit a logistic classifier on its embeddings:

$$\boxed{p(y = 1 \mid \boldsymbol{h}) = \sigma(\boldsymbol{w}^{\mathsf{T}} \boldsymbol{h} + b)}.$$

- Train only the 768 weights in $\boldsymbol{w}$ and the intercept b.
- Use a softmax head for more than two classes.
- Validate how well a *linear probe* predicts labels.

The features are learned by a deep model; the last classifier is linear.

Week 9 summary

1. **Linear score:** $w^\top x + b$ defines the boundary.
2. **Least squares:** simple fit, but squared loss is not a class loss.
3. **Perceptron:** correct mistakes through sequential updates.
4. **Logistic regression:** sigmoid + BCE + gradient descent.
5. **Practice:** choose losses, regularization, and validation carefully.

$$\boxed{\text{score} \longrightarrow \text{training loss} \longrightarrow \text{learned decision}}$$

Next week: support vector machines and margins.

A linear boundary can be learned through very different objectives

Questions and discussion